

AegisMesh: Self-Healing Autonomous Enterprise IoT Mesh Network

Comprehensive Technical & Academic Report

1. Introduction

The rapid expansion of the Internet of Things (IoT) has led to an exponential increase in connected devices in industrial, enterprise, and smart-city environments. Traditional hub-and-spoke network topologies struggle with single points of failure, scaling limitations, and high latency. **AegisMesh** is a next-generation, self-healing, autonomous IoT mesh networking platform designed to solve these challenges.

AegisMesh leverages ESP32 microcontrollers communicating via MQTT and WebSockets, managed by an intelligent Node.js backend. The platform provides a rich, web-based Network Operations Center (NOC) dashboard utilizing React.js, allowing administrators to monitor network topology in real-time. By implementing dynamic routing protocols (such as Dijkstra's shortest-path algorithm) and AI-driven predictive risk monitoring, AegisMesh ensures uninterrupted data flow even when individual nodes experience power loss, thermal events, or deliberate network attacks.

2. Literature Review

In recent years, Wireless Sensor Networks (WSNs) and IoT mesh networks have been extensively studied. Early protocols like Zigbee and Z-Wave introduced basic mesh capabilities but often suffered from bandwidth constraints and proprietary lock-in.

- **Dynamic Routing:** Research by Perkins et al. on Ad hoc On-Demand Distance Vector (AODV) routing highlighted the need for reactive routing in mobile networks. AegisMesh adapts similar principles but applies optimized shortest-path algorithms centrally for lower latency.
- **Self-Healing Networks:** Recent literature emphasizes "self-healing" properties in industrial IoT. Studies show that automated failover mechanisms can reduce network downtime by up to 98%.
- **Edge AI in IoT:** The integration of predictive analytics into network management represents the cutting edge of NOC software, moving from reactive alerts to proactive, AI-driven node isolation before cascading failures occur.

3. Problem Definition

Enterprise IoT deployments frequently suffer from physical obstructions (e.g., metal server racks, concrete walls) and environmental factors that degrade signal quality, leading to packet loss. When a critical routing node fails due to power loss or thermal overload, dependent edge nodes become disconnected, leading to data loss and system downtime.

Furthermore, network administrators lack real-time, visual, and predictive tools to monitor mesh health, forcing them to rely on archaic terminal logs or retroactive analysis to diagnose network failures. There is a critical need for a system that not only dynamically heals itself but also provides a comprehensive, 3D-visualized command center for human operators.

4. Objectives

The primary objectives of the AegisMesh project are:

1. **Develop a resilient mesh architecture:** Enable ESP32 hardware nodes to dynamically route packets through the network, bypassing failed or quarantined nodes instantly.
2. **Implement Millisecond Self-Healing:** Ensure that if a node drops off the network, the central gateway recalculates and propagates new Dijkstra-optimized routes within milliseconds.
3. **Build an Enterprise NOC Dashboard:** Create a high-fidelity, visually immersive React.js dashboard (Deep Space Command Center aesthetic) with real-time Cytoscape.js topology mapping.
4. **Integrate Predictive AI:** Deploy a predictive risk engine that monitors node telemetry (latency, CPU, packet loss) and calculates a failure probability score, allowing for proactive node isolation.
5. **Ensure Scalability and Hybrid Operation:** Support a seamless blend of live physical hardware and simulated nodes to allow for extensive testing, demonstration, and network scaling.

5. Methodology & System Architecture

The development of AegisMesh follows an iterative, full-stack Agile methodology, strictly decoupled into three primary domains: Edge Hardware, Control Plane Backend, and NOC Frontend.

5.1. Edge Hardware Layer (ESP32 Firmware)

The hardware nodes use ESP32 microcontrollers programmed in C++ (via PlatformIO/Arduino IDE).

- **Core Functions:**
 - `setup_wifi()`: Handles robust connection to the local access point.
 - `reconnect_mqtt()`: Exponential backoff mechanism ensuring persistent broker connection.
 - `publish_telemetry()`: Transmits localized sensor data (CPU load, temperature from DHT11, uptime, RAM usage, RSSI) to the broker under the `mesh/telemetry/{nodeId}` topic.
 - `on_route_update()`: Listens on `mesh/routes/{nodeId}` for incoming routing instructions from the Gateway. Updates the local routing table to forward packets to the correct neighbor node.

5.2. Control Plane Backend (Node.js & Aedes MQTT)

The backend acts as the intelligent Gateway for the mesh, brokering messages and calculating paths.

- **MQTT Broker (mqttService.js):**

Utilizes the aedes library to host a native MQTT server on port 1883. It authenticates nodes and parses payloads.

 - *Function `handleLiveHeartbeat(payload)`*: Registers new hardware connections, injecting them into the topology via `topologyService.addLiveNode()`. Updates the lastHeartbeat timestamp.
 - *Function `handleTelemetry(payload)`*: Ingests environmental telemetry and parses CPU/Thermal spikes. Triggers the AI predictive scoring model.
- **Topology & Routing Engine (topologyService.js):**
 - *Function `dijkstra(startNode, endNode)`*: An optimized implementation of Dijkstra's algorithm. Calculates the lowest-cost path based on a dynamic routingPolicy.
 - *Function `getLinkCost(from, to, baseLatency)`*: Adjusts edge weights dynamically. For instance, if `routingPolicy === 'energy'`, it penalizes routes passing through nodes with `battery < 20%`. If `routingPolicy === 'reliability'`, it multiplies cost by packet loss percentage.

- *Function recalculateRSSIAndLinks()*: Applies a Free Space Path Loss (FSPL) algorithm ($rssi = -30 - 20 * \text{Math.log10}(\text{meters})$). It also calculates line-segment intersections using `checkLineIntersection()` to simulate signal attenuation when Wi-Fi passes through virtual obstacles like "Concrete Partitions" (-18dBm) or "Server Racks" (-30dBm).
- *Function addLiveNode(metadata)*: Ingests an unknown hardware device, computes a random orbital coordinate mapping around the central gateway (using `Math.cos()` and `Math.sin()`), and integrates it into the mesh without overlapping existing nodes.

5.3. Frontend NOC Dashboard (React.js & Zustand)

The frontend provides a real-time, high-fidelity monitoring interface. It leverages framer-motion for advanced micro-animations and a Deep Space Command Center aesthetic (Cyan #00d4ff and Violet #8b5cf6).

- **State Management (store/meshStore.js):**
Uses Zustand to maintain an immutable global state.
 - *Function updateTopology(data)*: Replaces the node/edge arrays and recalculates global metrics (e.g., active routes, failed nodes, system health percentage).
 - *Function addEvent(event)*: Pushes a real-time log into the EventConsole, capping the list at 100 to prevent memory leaks.
- **Visualizer (NetworkGraph.jsx):**
Implements cytoscape.js for graph rendering.
 - Sub-components style nodes dynamically based on node.status (e.g., failed nodes turn red with 0.4 opacity, active nodes pulse green).
 - *Function runLayout()*: Executes the physics layout engine when new hardware joins.
- **Packet Animation (PacketCanvas.jsx):**
A high-performance HTML5 <canvas> overlay layered on top of Cytoscape.
 - *Function draw(ts)*: Uses requestAnimationFrame to draw glowing cyan particles (representing data packets) traversing the exact coordinates of active edges. Calculates speed dynamically based on edge latency weights.
- **Predictive AI Engine (PredictionCard.jsx):**
Displays the outputs of the backend's anomaly detection. Features a risk gauge and Estimated Time to Failure (ETF) based on exponential moving averages of node CPU and thermal spikes.
- **Hardware Integration Controls (SimulationPanel.jsx):**
Allows administrators to trigger deliberate failures (e.g., `socket.emit('node:fail', { nodeId: 'ESP32-A' })`) to observe the network's millisecond self-healing and rerouting in real-time.

6. Proposed Outcomes

- A robust, operational 4+ node ESP32 hardware mesh network capable of autonomous routing.
- A fully functional Node.js backend that handles routing policies (Latency-optimized, Energy-saver, Reliability).
- A visually stunning, production-ready React frontend featuring a live topology map, event streams, and predictive AI monitors.
- Demonstrated millisecond failover capabilities, where the physical disconnection of a node results in immediate route recalculation without data loss.
- A complete diagnostic suite proving the operational status of 15 advanced system features (including Network DVR replays, Geo-fencing, and Multi-mesh switching).

7. Timeline of the Project

Phase	Duration	Milestones
Phase 1: Architecture & Prototyping	Weeks 1-2	System design, protocol selection (MQTT), ESP32 firmware baseline, backend scaffolding.
Phase 2: Core Routing & Self-Healing	Weeks 3-4	Dijkstra algorithm implementation, heartbeat mechanisms, failover logic integration.
Phase 3: Frontend Development	Weeks 5-6	React dashboard setup, Cytoscape network graph, state management, WebSocket integration.
Phase 4: Advanced Features & AI	Weeks 7-8	Predictive risk engine, automated playbooks, routing policies, energy simulation.
Phase 5: UI Polish & Hardware Integration	Weeks 9-10	Premium "NOC" UI/UX overhaul, hardware deployment, stability testing, bug fixing.
Phase 6: Documentation & Presentation	Weeks 11-12	Final reports, PPT preparation, demonstration video recordings, final system audit.

8. Conclusion

AegisMesh represents a significant leap forward in enterprise IoT management. By combining the physical resilience of edge-computing mesh networks with the centralized intelligence of an AI-driven backend and a state-of-the-art visual dashboard, it solves the most pressing issues in modern IoT deployments: reliability, observability, and scale. The system not only reacts to failures in milliseconds but predicts them before they occur, ensuring absolute network integrity for mission-critical applications. The complete separation of concerns—from the C++ hardware level up to the React/Canvas visualization layer—demonstrates a highly scalable, production-grade architecture.

9. References

1. Perkins, C. E., & Royer, E. M. (1999). Ad-hoc on-demand distance vector routing. *Second IEEE Workshop on Mobile Computing Systems and Applications*.
2. Akyildiz, I. F., Su, W., Sankarasubramaniam, Y., & Cayirci, E. (2002). Wireless sensor networks: a survey. *Computer networks*, 38(4), 393-422.
3. Cytoscape Consortium. (2023). Cytoscape.js: Graph theory (network) library for visualization and analysis.
4. MQTT.org. (2023). MQTT: The Standard for IoT Messaging.
5. Espressif Systems. (2023). ESP32 Series Datasheet and Technical Reference Manual.